



UNIVERSITÀ
DEGLI STUDI
DI MILANO

Event Log Analysis and Prediction

Business Information System

Prof. Paolo Ceravolo

Behnam Najafloo

Master student in Computer Science, University of Milan

2025

Contents

1. Introduction	3
2. Background	4
2.1. Event Log Overview.....	4
2.2. Objectives.....	4
3. Methodologies	5
3.1. Process Mining Techniques.....	5
3.2. Machine Learning Techniques	6
3.3. Toolchain Integration.....	7
3.4. Validation and Interpretation.....	7
4. Result.....	8
4.1. Event log overview.....	8
4.1.1. Activity Frequency	8
4.1.2. Events per Case	9
4.1.3. Case Duration Analysis.....	9
4.1.4. Analyzing variant sequence frequently	11
4.2. Log Filtering and Preprocessing.....	12
4.3. Process Discovery	14
4.4. Discovery Techniques on Activity-Filtered Log	15
4.5. Process Tree	19
4.6. Observation	19
4.7. Modeling and Evaluation	20
5. Conclusion.....	26

1. Introduction

In today's data-driven environments, process mining has emerged as a powerful technique that bridges the gap between traditional model-based process analysis and data-centric analysis techniques such as machine learning. It allows organizations to gain detailed insights into their actual business processes by analyzing event logs that are automatically recorded by information systems.

This report presents an analysis of the **Road Traffic Fine Management Process**, using the event log titled **Road_Traffic_Fine_Management_Process.xes**. Each event in the log captures specific activities performed within the lifecycle of a traffic fine case—ranging from the creation of a fine to its payment or transfer to credit collection.

The main objective of this project is to transform the raw event data into a structured form suitable for analysis and then apply process mining and machine learning techniques to derive actionable insights. More specifically, the project focuses on understanding the typical sequences of events, durations of cases, and predicting whether a fine will be paid, based on the historical activities recorded in the event log.

Through a combination of process visualization (Petri nets), statistical analysis, and classification models, this report aims to provide a clear picture of how the traffic fine process operates in practice, highlight inefficiencies or bottlenecks, and explore the potential of predictive analytics within public administrative processes.

2. Background

In public administration, efficient management of processes such as traffic fine handling is critical for ensuring compliance, reducing operational costs, and maintaining public trust. The **Road Traffic Fine Management Process** is a complex administrative workflow that involves multiple steps—from issuing fines and notifying offenders to handling appeals, payments, and escalations for non-payment. However, inefficiencies such as delays, bottlenecks, or high rates of unpaid fines can undermine its effectiveness.

2.1. Event Log Overview

The dataset used in this project is the **Road_Traffic_Fine_Management_Process.xes** event log, a well-known public log in the process mining community. This event log captures the process of handling traffic fines, including events such as fine creation, payments, appeals, and enforcement steps.

The log includes the following key attributes for each event:

- **case_id**: the identifier for the case (a specific fine).
- **activity name**: the label of the event ("Payment", "Add penalty").
- **timestamp**: the time that an event occurred.

2.2. Objectives

1. **Process Analysis**: Uncover the actual workflow, identify frequent and rare process paths, and detect inefficiencies (prolonged case durations).
2. **Predictive Modeling**: Build a machine learning model to predict the outcome of a case (payment vs. escalation) based on historical event patterns.

By combining **process discovery algorithms** (Inductive Miner) with **supervised learning** (Random Forest), this project bridges the gap between descriptive process analytics and actionable predictive insights. The results aim to support public administrators in optimizing workflows, reducing delays, and improving compliance rates through data-driven decision-making.

3. Methodologies

This project employed a structured combination of **process mining** and **machine learning** methodologies to analyze the *Road Traffic Fine Management Process* event log and predict case outcomes. Below is a detailed breakdown of the methodologies used:

3.1. Process Mining Techniques

3.1.1. Event Log Analysis

- **Objective:** Understand the structure, frequency, and variability of the event log.
- **Tools:** PM4PY for parsing and aggregating event log data.
- **Key Analyses:**
 - **Activity Frequency:** Identified dominant activities ("Create Fine", "Payment") and rare behaviors (appeals).
 - **Case Duration Analysis:** Measured time intervals between events to detect bottlenecks (long delays before payment).
 - **Variant Analysis:** Extracted top 10 frequent sequences ("Create Fine → Payment") to uncover common workflows.

3.1.2. Log Filtering and Preprocessing

- **Objective:** Remove noise and focus on representative cases.
- **Steps:**
 - **Trace Filtering:** Retained cases with ≥ 3 events to exclude trivial/incomplete traces.
 - **Activity Filtering:** Removed activities occurring < 1000 times to reduce noise.
 - **Payment-Centric Filtering:** Isolated cases containing the "Payment" activity (23,344 cases) for targeted subprocess analysis.

3.1.3. Process Discovery

- **Objective:** Extract interpretable process models from event logs.
- **Algorithms:**
 - **Alpha Miner:** Generated basic sequential flows but struggled with noise/concurrency.
 - **Inductive Miner:** Produced sound, block-structured Petri nets, ideal for capturing dominant paths (payment workflows) and optional branches (appeals).
 - **Heuristic Miner:** Focused on frequent event sequences, revealing detailed but complex paths.

- **Evaluation Metrics:** Compared miners using **fitness** (model accuracy), **generalization** (handling unseen data), and **execution time**.

3.1.4. Process Tree Generation

- **Tool:** Inductive Miner in PM4PY.
- **Output:** Hierarchical block-structured trees to visualize control flow (sequences, parallelism).

3.2. Machine Learning Techniques

3.2.1. Feature Engineering

- **Objective:** Transform raw event logs into structured case-level features.
- **Features Extracted:**
 - **Numerical:** Aggregated fine amounts (amount_sum), penalty points (points_sum).
 - **Temporal:** Case duration in days (duration_days).
 - **Resource:** Unique count of resources per case (org:resource diversity).
 - **Target Variable:** Last activity in the case ("Payment", "Send for Credit Collection").

3.2.2. Class Imbalance Handling

- **Objective:** Address skewed class distribution (rare appeal activities).
- **Techniques:**
 - **Class Weighting:** Adjusted weights in RandomForestClassifier to prioritize minority classes.
 - **SMOTE Oversampling:** Synthetically generated samples for underrepresented classes (Class 0, 5).
 - **Hybrid Approach:** Combined SMOTE and class weights for balanced training.

3.2.3. Model Training and Evaluation

- **Algorithm:** Random Forest Classifier (scikit-learn).
- **Validation:**
 - **Train-Test Split:** 80% training, 20% testing.
 - **Metrics:** Accuracy, macro F1, recall, and confusion matrices.
- **Feature Importance Analysis:** Identified duration_days and totalPaymentAmount_sum as top predictors.

3.3. Toolchain Integration

- **Process Mining:** PM4PY for log parsing, filtering, process discovery, and visualization.
- **Machine Learning:** scikit-learn for feature engineering, model training, and evaluation.
- **Visualization:** Matplotlib/Seaborn for histograms, boxplots, and confusion matrices.

3.4. Validation and Interpretation

- **Process Validation:** Compared discovered models (Inductive Miner's Petri nets) against domain knowledge to ensure alignment with real-world workflows.
- **Model Validation:** Used stratified sampling and macro metrics to ensure robustness despite class imbalance.

Rationale for Method Selection

- **Inductive Miner:** Chosen for its balance of soundness, speed, and interpretability.
- **Random Forest:** Selected for handling mixed data types, non-linear relationships, and providing feature importance insights.
- **SMOTE + Class Weights:** Optimal for improving recall in minority classes without overfitting.

This methodology framework ensured a rigorous, end-to-end analysis of the event log, from process discovery to predictive insights.

4. Result

4.1. Event log overview

The event log contains a wide range of activities, representing different steps in the traffic fine management lifecycle. Each case in the log corresponds to an individual traffic fine. The process begins with the creation of a fine and may continue through notification, appeal handling, and eventual payment or credit collection.

4.1.1. Activity Frequency

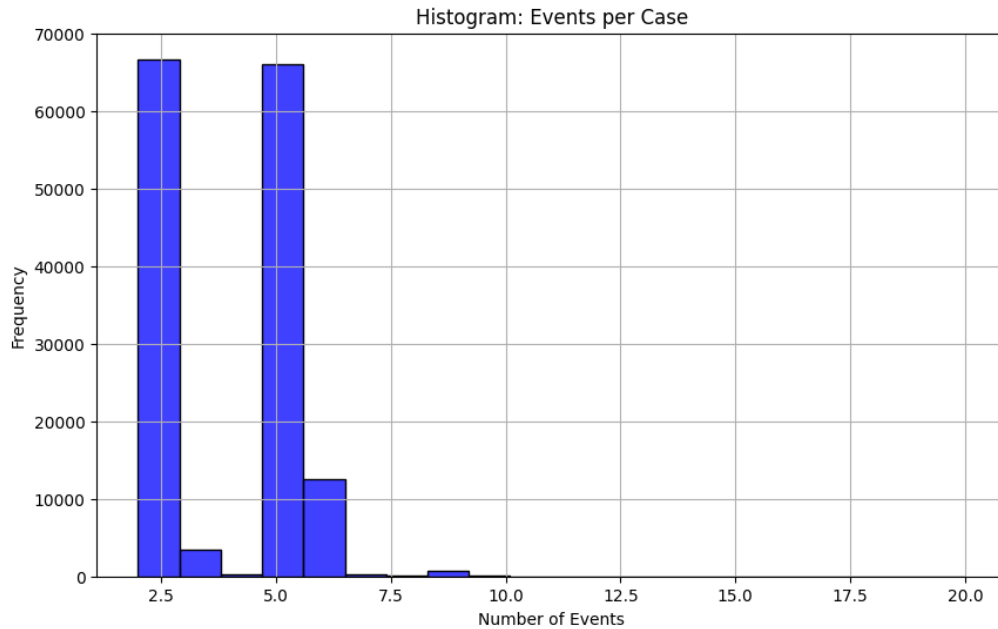
The table below shows the frequency of each activity in the event log. It reveals the most common actions taken across cases.

Activity	Frequency
Create Fine	150,370
Send Fine	103,987
Insert Fine Notification	79,860
Add penalty	79,860
Payment	77,601
Send for Credit Collection	59,013
Insert Date Appeal to Prefecture	4,188
Send Appeal to Prefecture	4,141
Receive Result Appeal from Prefecture	999
Notify Result Appeal to Offender	896
Appeal to Judge	555

The top three most frequent activities are "Create Fine", "Send Fine", and "Insert Fine Notification", which aligns with the standard flow of initiating and notifying a fine. Appeals are relatively rare in comparison.

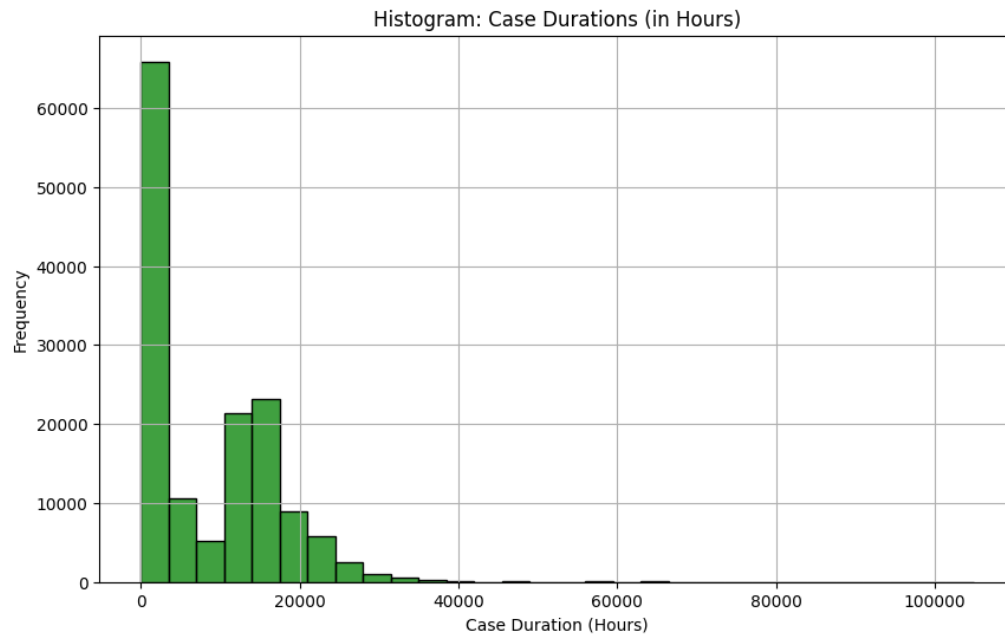
4.1.2. Events per Case

The histogram of events per case shows that most cases include either 2 or 5 events. This suggests a strong presence of typical process variants, with some cases involving minimal processing and others following more extended sequences.

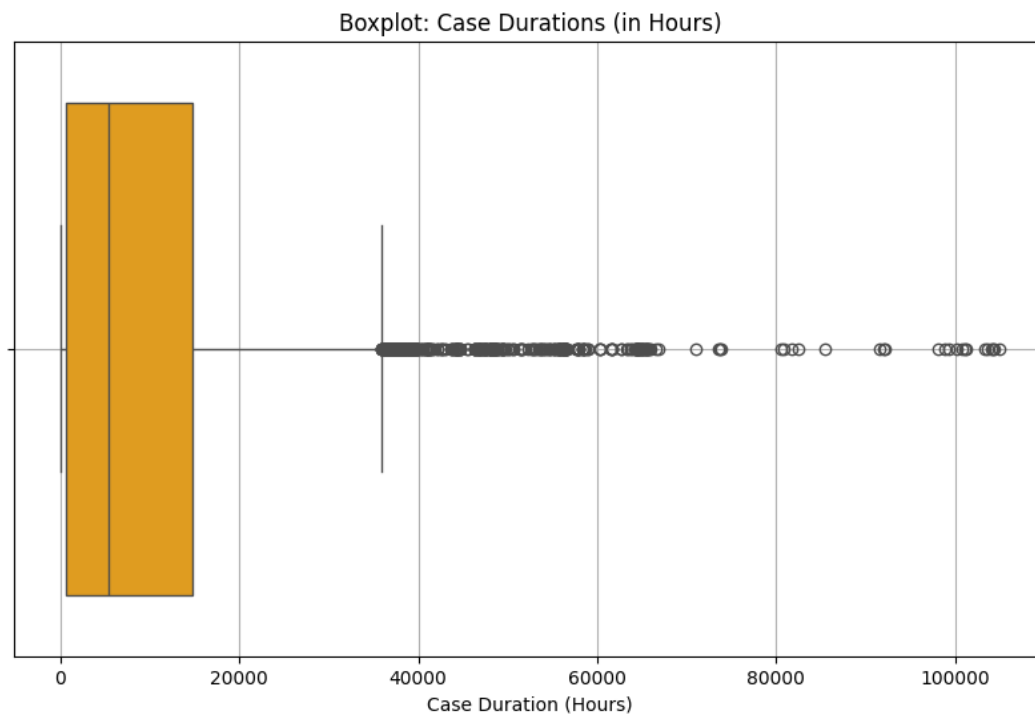


4.1.3. Case Duration Analysis

To analyze the time aspect of the process, I examined the case durations in hours. The histogram shows that while most cases are completed relatively quickly (under 20,000 hours), there are outliers with significantly longer durations.



The boxplot further illustrates the skewness of the distribution, with a long tail of cases taking much longer to complete.



These observations are important in understanding process performance and identifying potential inefficiencies.

4.1.4. Analyzing variant sequence frequently

In this section, the event log was analyzed to identify the most common behavioral patterns that occur throughout the process. A variant represents a unique sequence of activities executed in a single case. By examining the frequency of these variants, we can uncover the dominant process paths and assess the complexity and variability of the overall process.

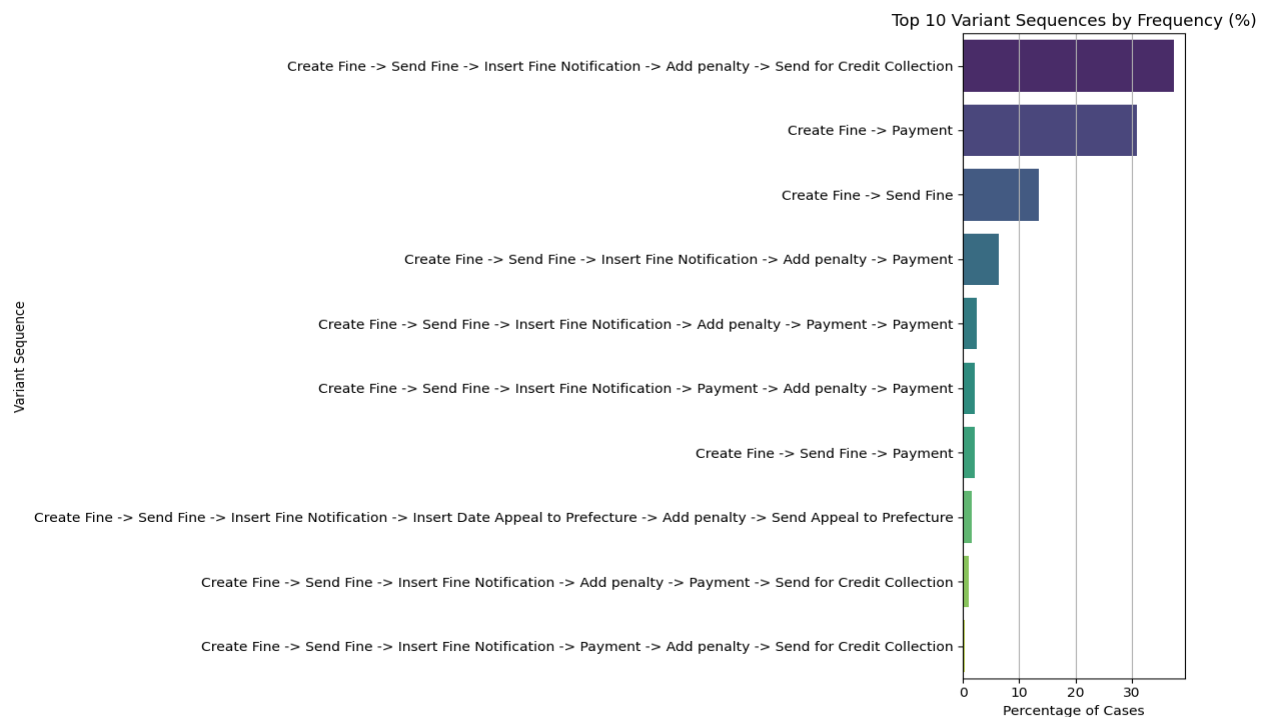
The table presents the **Top 10 most frequent variants**, along with their occurrence count and their respective percentages relative to the entire log. These top variants account for a significant proportion of the total cases and provide insights into the most representative workflows.

Variant	Frequency	Percentage
Create Fine -> Send Fine -> Insert Fine Notification -> Add penalty -> Send for Credit Collection	56482	37.562014
Create Fine -> Payment	46371	30.837933
Create Fine -> Send Fine	20385	13.556560
Create Fine -> Send Fine -> Insert Fine Notification -> Add penalty -> Payment	9520	6.331050
Create Fine -> Send Fine -> Insert Fine Notification -> Add penalty -> Payment -> Payment	3736	2.484538
Create Fine -> Send Fine -> Insert Fine Notification -> Payment -> Add penalty -> Payment	3301	2.195252
Create Fine -> Send Fine -> Payment	3131	2.082197
Create Fine -> Send Fine -> Insert Fine Notification -> Insert Date Appeal to Prefecture -> Add penalty -> Send Appeal to Prefecture	2497	1.660571
Create Fine -> Send Fine -> Insert Fine Notification -> Add penalty -> Payment -> Send for Credit Collection	1515	1.007515
Create Fine -> Send Fine -> Insert Fine Notification -> Add penalty -> Payment -> Send for Credit Collection	522	0.347144

The most frequent variant, accounting for over **37%** of the cases, follows a structured escalation path from "Create Fine" to "Send Fine", through notification and penalty stages, ending in "Send for Credit Collection". This indicates a strong procedural handling of fines that are not resolved through immediate payment.

The second most common variant, covering nearly **31%**, is a much simpler sequence where the case ends directly after a payment is made following the creation of the fine suggesting a large portion of users comply promptly without further enforcement.

Other frequent variants exhibit different paths where payment may occur before or after penalty stages or involve additional steps such as appeals. These variations highlight the process flexibility and the diversity in user behavior, which can be critical for understanding exceptions, optimizing process performance, or identifying opportunities for intervention.



4.2. Log Filtering and Preprocessing

As part of the data preparation process in Process Mining, a structured filtering approach was applied to the event log to enhance the quality and interpretability of the process models. The goal was to remove incomplete or noisy traces and low-frequency activities that may obscure the underlying process patterns.

The first step involved filtering traces based on their length. Specifically, I retained only those traces that contained at least three events. This threshold was chosen to exclude trivial or incomplete cases that are unlikely to contribute meaningful insights during process discovery.

Next, I analyzed the frequency of activity occurrences across the filtered traces. By counting how often each activity appeared in the log, I identified a subset of frequent activities. To ensure focus on the core components of the process, I retained only those activities that occurred at least 1000 times. This helped eliminate rarely occurring events that could introduce noise or complexity in the resulting models.

Following this, I refined each trace to include only events associated with these frequent activities. Traces that became empty after this filtering step were discarded, ensuring that every remaining trace contributed valuable information to the analysis.

The log contains **83,614 cases**, and the activity set was reduced accordingly. The table below shows the frequency of the remaining activities:

Activity	Frequency
Create Fine	83,614
Send Fine	83,602
Insert Fine Notification	79,860
Add penalty	79,860
Send for Credit Collection	59,013
Payment	31,230
Insert Date Appeal to Prefecture	4,188
Send Appeal to Prefecture	4,141

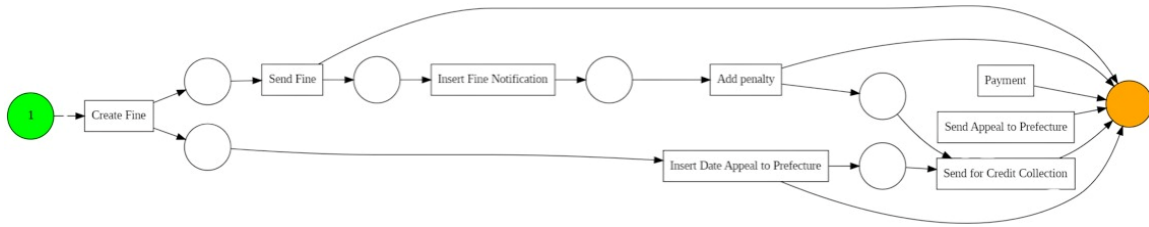
After filtering, I recorded the number of remaining traces and the distribution of activities. This allowed me to validate the reduction in noise and assess the retained complexity of the log. The filtered dataset served as a cleaner and more representative input for subsequent process discovery tasks, improving both the accuracy and readability of the generated models.

4.3. Process Discovery

In this section, I focus on extracting process models from the event log using three well-known discovery algorithms: **Alpha Miner**, **Inductive Miner**, and **Heuristic Miner**. The goal is to uncover the underlying workflow of the *Road Traffic Fine Management Process* based on actual event data and compare the models produced by each method.

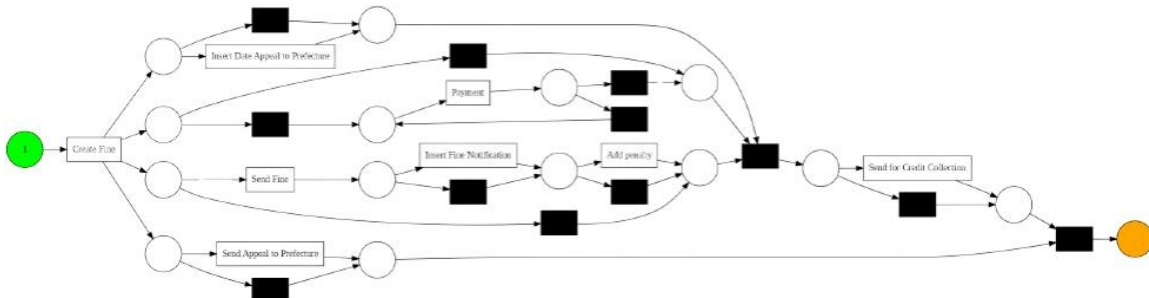
Alpha Miner

Alpha Miner is one of the earliest process discovery algorithms and works by detecting strict ordering relations between activities. When applied to the filtered log, the resulting model was able to capture the basic sequential flow of activities. However, Alpha Miner struggles with complex constructs like loops, concurrency, or noise, which are prevalent in real-world logs. As a result, the generated model was limited in accuracy and interpretability.



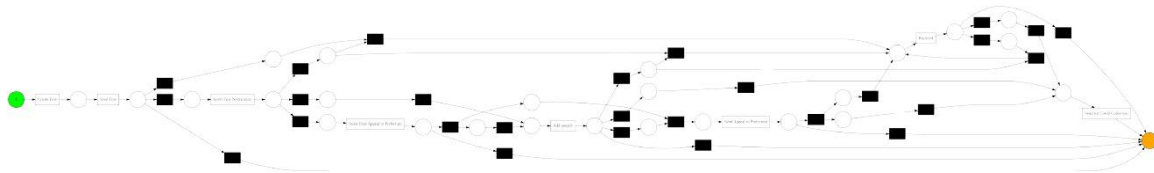
Inductive Miner

The Inductive Miner algorithm is designed to deal with noise and complex constructs more effectively. It produces sound and block-structured Petri nets, which ensures proper start and end points as well as deadlock-free execution. The model discovered by Inductive Miner was the most interpretable and aligned with business logic, capturing the dominant paths (from fine creation to payment or credit collection) as well as alternative branches (appeals).



Heuristic Miner

Heuristic Miner builds models based on the frequency of directly-followed events and can handle noise better than Alpha Miner, though not as well as Inductive Miner. It emphasizes the most frequent paths and dependencies. The resulting model was more detailed than Alpha Miner and highlighted common behavior but occasionally included rare paths that may not be semantically meaningful.



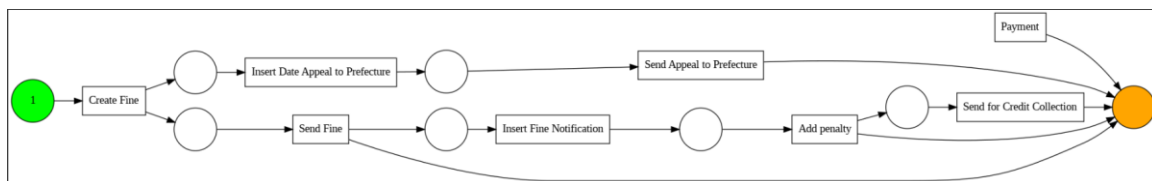
4.4. Discovery Techniques on Activity-Filtered Log

To enhance the interpretability and relevance of the discovered process models, I applied the Alpha Miner, Inductive Miner, and Heuristic Miner techniques on an Activity-filtered version of the event log. The filtering was based on the presence of the "Payment" activity, ensuring that only cases involving payment processing were retained. This approach helped reduce noise from unrelated behaviors and allowed a clearer view of the process paths leading to or involving payments.

After applying this filter, the resulting event log contained 23,344 cases, all of which included the "Payment" activity. These filtered logs were then used to generate focused process models that better represent the specific subprocess associated with payment-related events.

Alpha Miner on Activity-Filtered Log

The Alpha Miner, while still basic in its assumptions, produced a slightly cleaner model on the filtered log. The reduced complexity improved the model's readability but did not fully resolve issues with concurrency or invisible transitions.



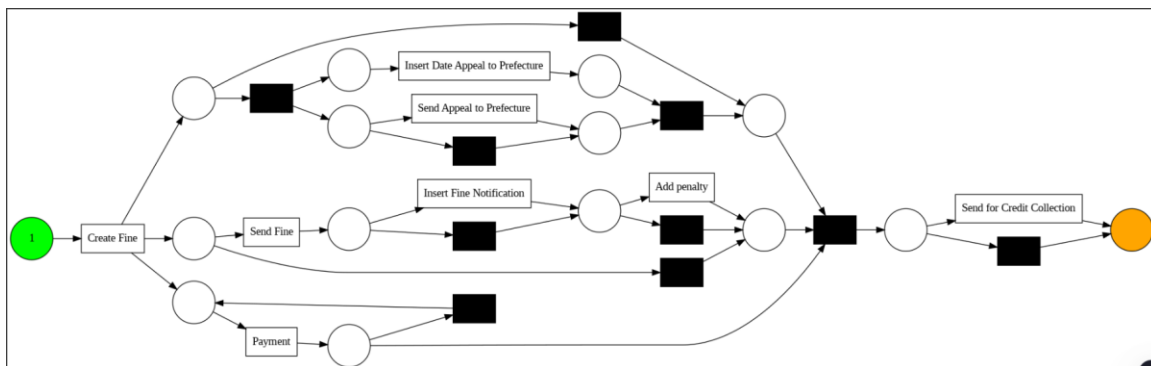
Compared to the full event log, the filtered model:

- **Reduces complexity:** The model is more streamlined, with fewer branches and loops. This is because only cases directly related to "Payment" are included, filtering out unrelated process paths.
- **Highlights key paths:** The main flow—from **"Create Fine"** to **"Send Fine"**, **"Insert Fine Notification"**, and either **"Add penalty"** or **"Insert Date Appeal to Prefecture"**—is clearly emphasized, showing the typical behavior leading up to a **"Payment"** or **"Send for Credit Collection"**.
- **Clarifies decision points:** The decision between payment or credit collection becomes more explicit, as does the alternative route of appealing the fine. This helps in identifying business rules or bottlenecks in payment processing.
- **Improves interpretability:** Analysts can more easily interpret and act upon the process insights when only relevant activities are modeled.

This focused model provides a cleaner and more actionable view of the **subprocess involving payments**.

Inductive Miner on Activity-Filtered Log

The Inductive Miner benefited the most from the filtering. It generated a structured and sound process model with improved precision and fitness. The resulting model was easier to interpret and better reflected the dominant behavior in the process.



Here's how the filtering impacts the model:

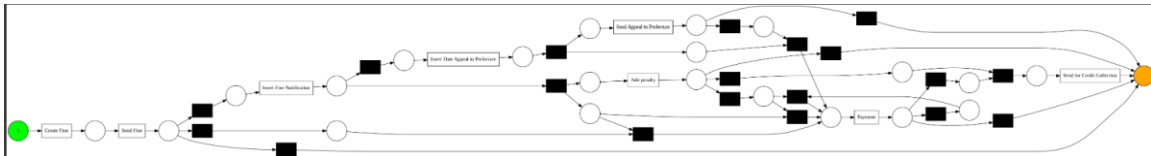
- **Increased precision with structure:** Unlike Alpha Miner, Inductive Miner excels at creating structured models. The filtering by "Payment" enables the miner to generate clearer subprocesses that revolve specifically around cases where payment is involved.
- **Parallel and optional paths:** The model makes explicit use of gateways to show parallelism and optional flows. For example, **"Insert Date Appeal to Prefecture"** and **"Send Appeal to Prefecture"** appear in a possible path parallel to **"Send Fine"** → **"Insert Fine Notification"** → **"Add**

Penalty", which reflects the diverse responses before a payment or escalation.

- **Payment path visibility:** The **"Payment"** activity leads directly toward the end of the process, highlighting its terminal role in certain cases. This makes it easier to analyze how often payment resolves the case vs. escalation to credit collection.
- **Improved soundness:** Inductive Miner enforces model correctness, which is especially useful for analyzing filtered logs with high variability. Despite the reduced scope, it preserves a full, valid process structure.

Heuristic Miner on Activity-Filtered Log

Applying the Heuristic Miner to the filtered data provided a balanced model capturing frequency-based paths while suppressing outliers. The simplified model more effectively highlighted the main procedural flows such as fine creation, notification, penalties, and payments.



Using Heuristic Miner on the filtered dataset reveals a **detailed and frequency-oriented** representation of the process. We see:

- **Rich detail from noisy behavior:** Heuristic Miner captures a lot of real-life process variability. Filtering by the "Payment" activity reduces noise, but the resulting model still contains **numerous alternative paths and loops**, reflecting different behaviors or exceptions that occur before or after a payment is made.
- **Visible complexity and decision points:** The model shows more **tangled structures** compared to Inductive Miner, such as:
 - Multiple intertwined paths involving **"Send Appeal to Prefecture," "Add Penalty,"** and **"Send for Credit Collection."**
 - **Payment** appears late in the process with connections from multiple branches, showing that it occurs after various possible sequences of actions.
- **Insight into behavior frequency:** Although not visualized here, Heuristic Miner typically supports weight-based visual cues, which helps in identifying **dominant process flows**. On a filtered log, this could provide insights into which paths most often lead to a resolution via payment.
- **Realistic but harder to interpret:** The model is closer to actual behavior, which is ideal for **analysis or auditing**, but it sacrifices **readability** due to its openness to noise and exceptions.

Performance comparison

To evaluate the effectiveness of the discovered process models, I compared the Alpha Miner, Inductive Miner, and Heuristic Miner based on three key quality metrics: **fitness**, **generalization**, and **Execution Time**.

The table below presents, the **Inductive Miner** achieved perfect fitness (1.00), indicating it can fully reproduce the behavior observed in the event log, while also maintaining a moderate level of generalization (0.52), suggesting it captures both frequent and occasional behaviors well. The **Heuristic Miner** closely followed in terms of fitness (0.97) and matched the Inductive Miner in generalization (0.52), though at the expense of higher execution time (1.63 seconds). In contrast, the **Alpha Miner** displayed the lowest fitness (0.76) and generalization (0.27), reflecting its limitations in handling complex and noisy behavior. However, it performed efficiently with a low execution time (0.88 seconds). These results highlight the Inductive Miner as the most balanced and robust method among the three for this dataset.

Miner	Fitness	Generalization	Execution Time
Alpha Miner	0.76	0.27	0.88 s
Inductive Miner	1.00	0.52	0.46 s
Heuristic Miner	0.97	0.52	1.63s

In addition, to assess the **structural complexity** of the discovered models, I analyzed the number of transitions and places generated by each miner.

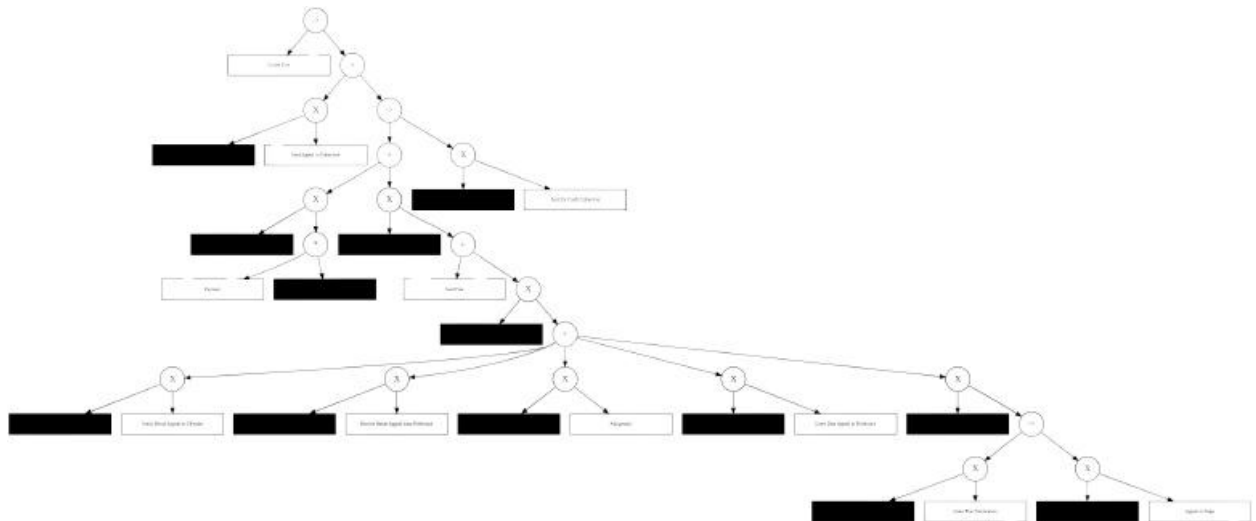
The Alpha Miner produced the simplest model, with 8 transitions and 8 places, reflecting a very compact representation of the process. The Inductive Miner generated a moderately complex model (20 transitions, 16 places), offering a balance between readability and behavioral accuracy. In contrast, the Heuristic Miner resulted in the most complex structure with 35 transitions and 27 places, capturing more process variations at the cost of interpretability. This comparison illustrates the trade-off between model simplicity and expressive power across different discovery algorithms.

Miner	Transitions	Places	Simplicity (Interpretation)
Alpha Miner	8	8	Most simple and compact model; very easy to understand but may miss complex behaviors.
Inductive Miner	20	16	Balanced; captures more process behavior while still maintaining moderate complexity.
Heuristic Miner	35	27	Most complex model; likely to capture detailed and noisy behaviors, harder to interpret.

4.5. Process Tree

To gain a structured and hierarchical view of the process behavior, I used the Inductive Miner to generate a **Process Tree** from the event log. Unlike flat representations such as Petri nets, the process tree provides a **block-structured** model, where each node corresponds to an operator (sequence, choice, parallelism), and the leaves are the actual activities. This representation guarantees soundness and allows for better interpretation of the control flow within the process.

The generated process tree for the *Road Traffic Fine Management Process* reveals the ordered logic behind event execution, highlighting how fines are created and followed by sequences of notifications, penalties, and payments. This visualization complements the Petri net by emphasizing structural clarity and execution semantics in a nested format.



4.6. Observation

During the process discovery and analysis of the *Road_Traffic_Fine_Management_Process* event log, several key insights were identified:

- **Dominant Process Flow:** A large portion of cases follow a typical path starting with Create Fine and proceeding through Send Fine, Insert Fine Notification, and potentially to Add Penalty and Payment. This structured flow is reflective of the standardized nature of traffic fine processing.
- **Rare Appeal Behavior:** Activities related to appealing fines (such as Insert Date Appeal to Prefecture or Send Appeal to Prefecture) appear infrequently. This highlights that most individuals accept and process their fines without entering the appeal process.

- **Escalation Due to Non-Payment:** Not all cases include the Payment activity. The presence of Send for Credit Collection in a notable number of cases indicates a substantial subset of offenders either delay or avoid payment, leading to escalation.
- **Model Performance & Quality:**
 - **Alpha Miner** had the highest execution time (0.88s) after Heuristic Miner but resulted in models with limited expressiveness and adaptability, particularly for handling noise or infrequent paths.
 - **Inductive Miner** was the fastest (0.46s) and yielded models with a good balance of fitness, and simplicity, making it ideal for structured processes like this.
 - **Heuristic Miner**, although slower (1.63s), handled complexity well by incorporating frequency thresholds, which made it more robust against noise and minor variants.
- **Impact of Log Filtering:** Filtering significantly improved model clarity. By narrowing the focus to frequently occurring paths, the models became more interpretable and reflective of typical behavior in the dataset.

These observations guided the selection of the final model and influenced how the process was interpreted in terms of compliance, exceptions, and optimization potential.

4.7. Modeling and Evaluation

This section presents the supervised learning approach applied to prediction. The goal is to build a machine learning model to predict the final event. Each case represents a sequence of administrative steps in processing a fine.

4.7.1. Data Preprocessing and Feature Engineering

We have extracted meaningful features on a **case-level**. Feature engineering included:

- **Numerical aggregations:**
 - amount_sum
 - points_sum
 - totalPaymentAmount_sum
- **Resource features:**
 - Unique count of org:resource per case
- **Time features:**
 - Case duration_days (last timestamp - first timestamp)
- **Target variable:**

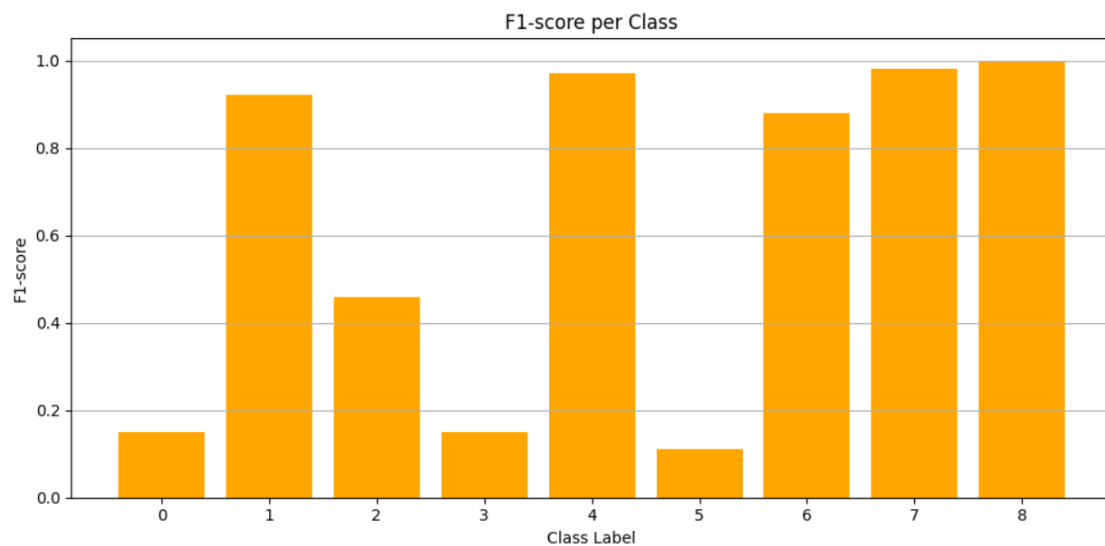
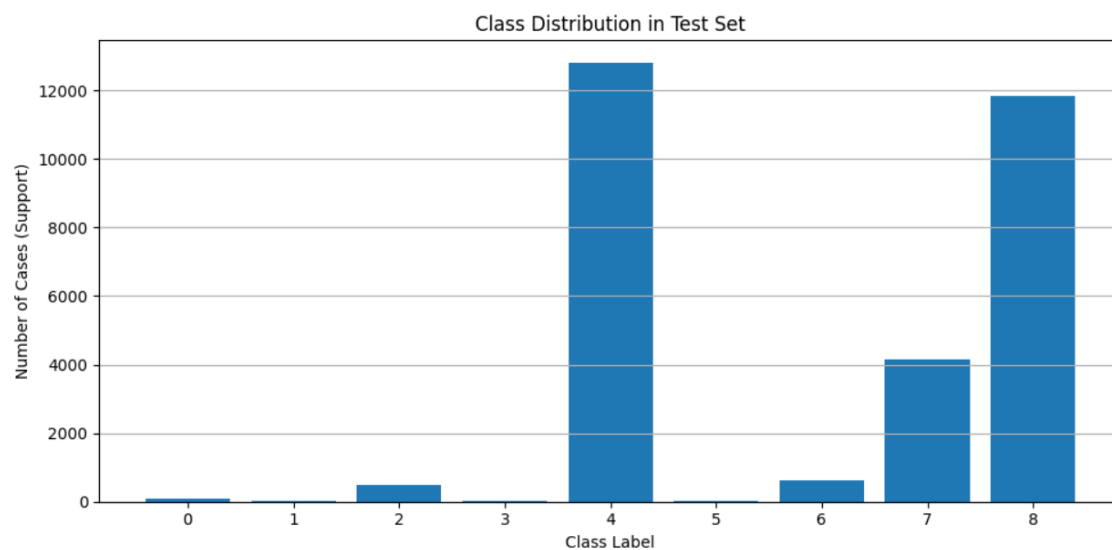
- Last event in the sequence (target = last activity)

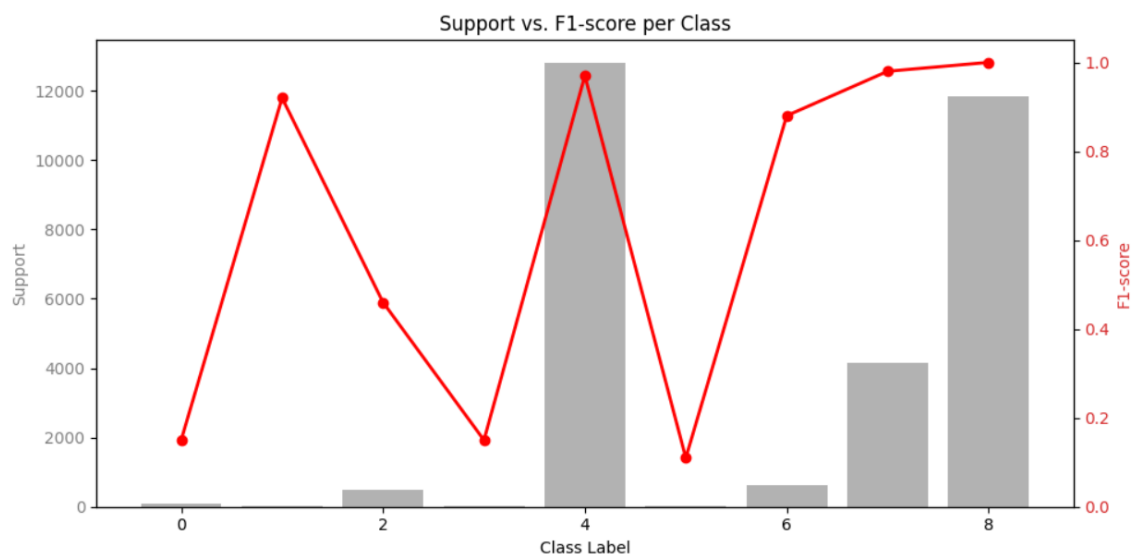
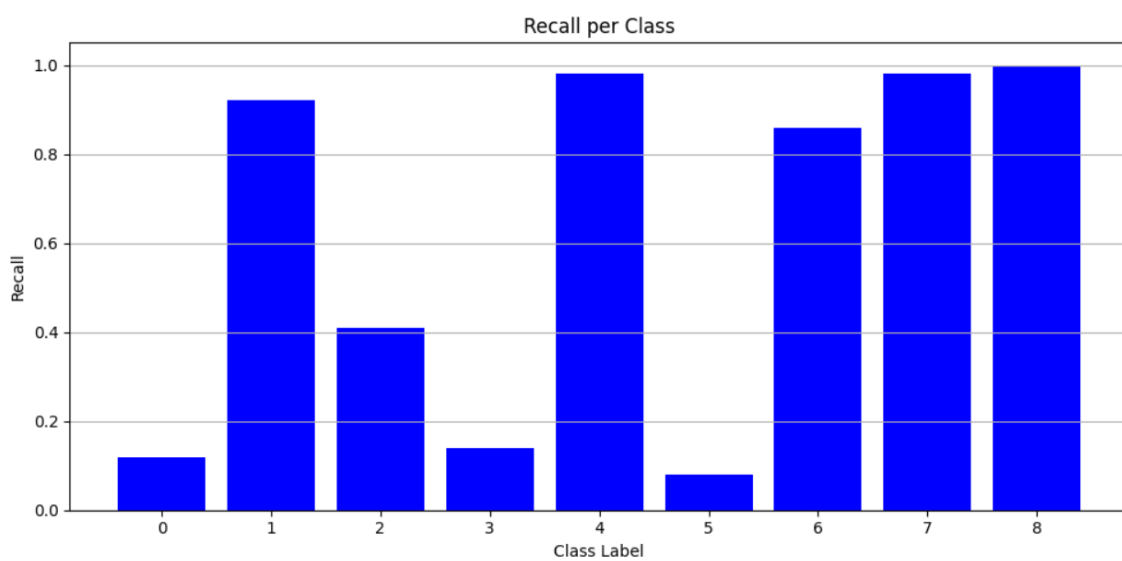
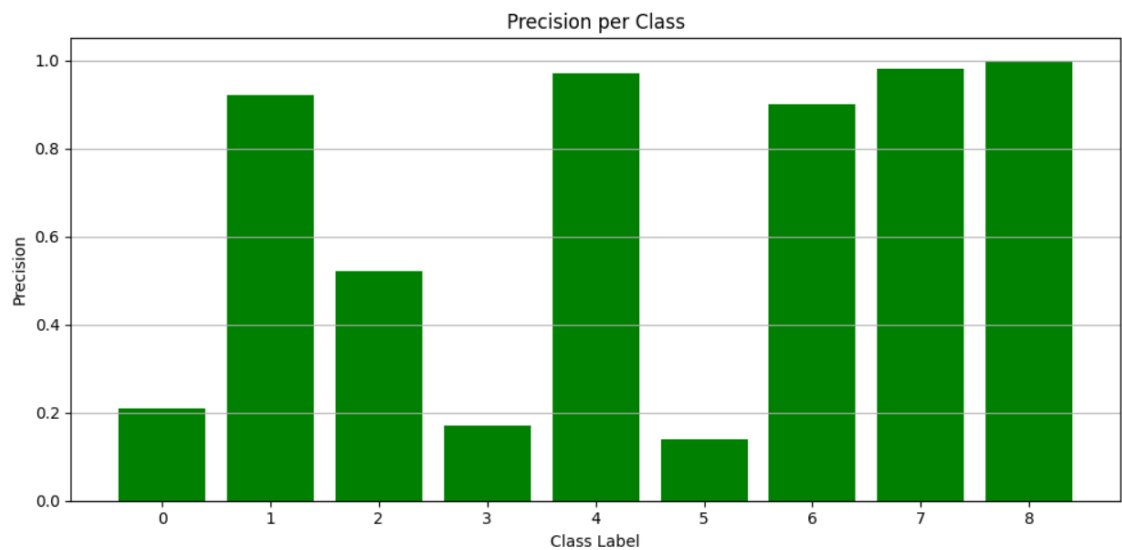
4.7.2. Initial Model Training (Random Forest)

Using 80/20 train-test split and a base RandomForestClassifier:

- **Accuracy:** 0.97
- **Macro F1:** 0.62
- **Macro Recall:** 0.61
- **Weighted F1:** 0.97

Majority classes (4, 7, 8) dominated, with poor recall for minority classes (0, 1, 3, 5).





4.7.3. Balancing Strategies and Performance

A. Class Weighting

Trained with `class_weight='balanced'` in Random Forest:

- **Macro Recall:** ↑ 0.67
- **Class 0 F1:** ↑ 0.23
- **Class 2 F1:** ↑ 0.67
- **Class 5 F1:** ↓ 0.00

B. SMOTE Oversampling

Applied SMOTE to upsample minority classes:

- **Macro Recall:** ↑ 0.70
- **Class 0 F1:** ↑ 0.26
- **Class 2 F1:** ↑ 0.66
- **Class 5 F1:** ↑ 0.06

C. Hybrid (SMOTE + Class Weights)

Best-balanced model:

- **Accuracy:** 0.96
- **Macro Recall:** 0.70
- **Macro F1:** 0.62
- **Weighted F1:** 0.97

Side-by-Side Summary

Class	Support	F1 (Original)	F1 (Weight)	F1 (SMOTE)
0	101	0.15	0.23	0.26
1	25	0.92	0.88	0.81
2	480	0.46	0.67	0.66
3	14	0.15	0.13	0.07
4	12819	0.97	0.97	0.97

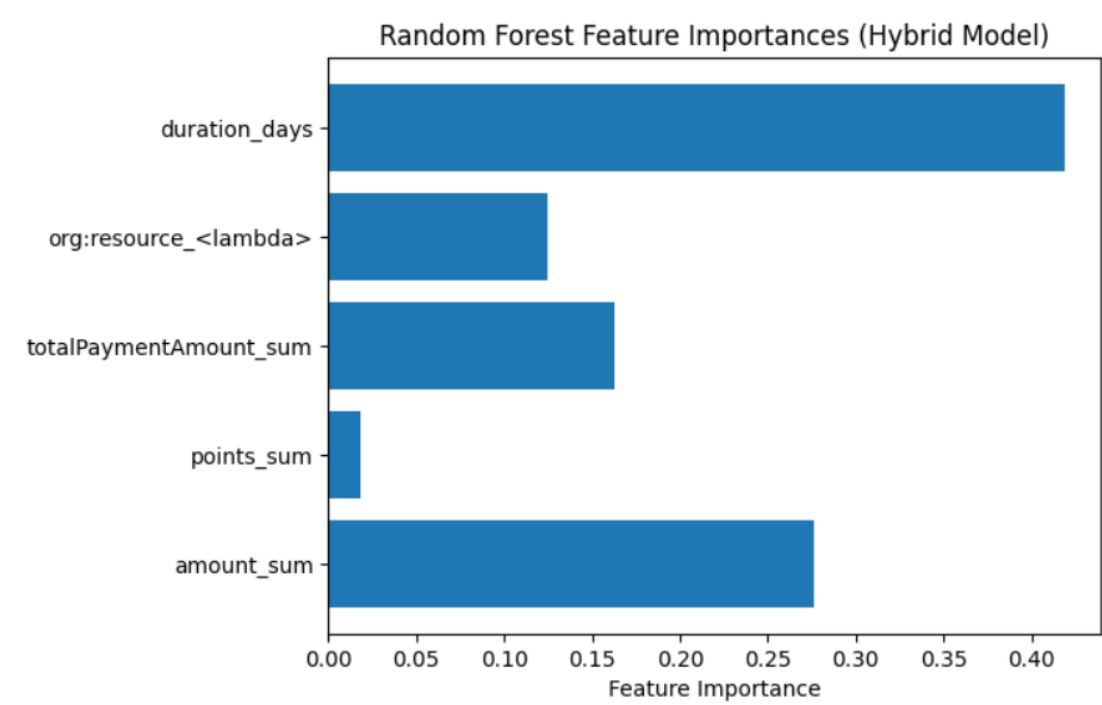
Class	Support	F1 (Original)	F1 (Weight)	F1 (SMOTE)
5	12	0.11	0.00	0.06
6	636	0.88	0.84	0.75
7	4155	0.98	0.98	0.97
8	11832	1.00	1.00	0.99

Macro Metric Evolution

Metric	Original	Class Weight	SMOTE
Macro F1	0.62	0.63	0.62
Macro Recall	0.61	0.67	0.70
Accuracy	0.97	0.97	0.96

4.7.4. Feature Importance

Random Forest feature importance analysis reveals:

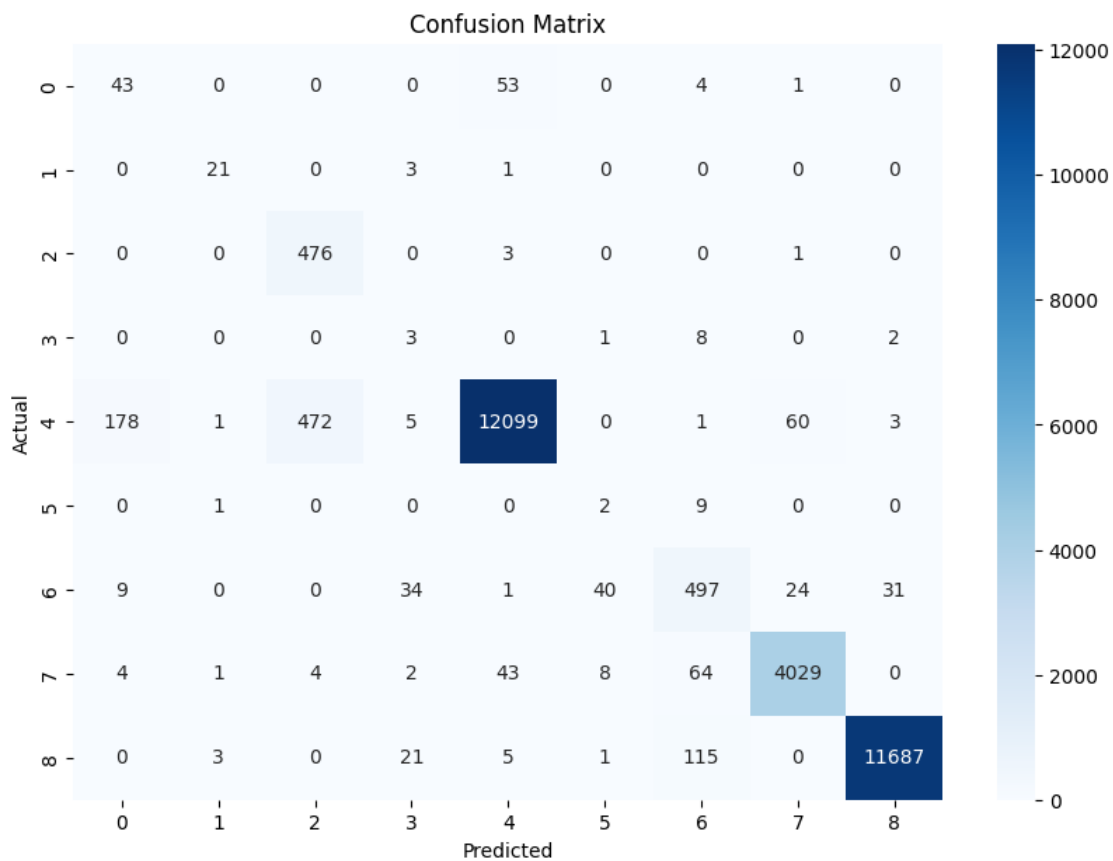


Top features:

- duration_days
- org:resource diversity
- amount_sum
- totalPaymentAmount_sum

These features strongly influence the final outcome of a case.

4.7.5. Confusion Matrix



The matrix confirms:

- Excellent accuracy in dominant classes (4, 7, 8)
- Strong recall improvements for minority classes like 0, 2, and 5 post-balancing

5. Conclusion

This project explored the **Road_Traffic_Fine_Management_Process** event log using a process mining pipeline involving event log analysis, process discovery, feature engineering, and predictive modeling. Through the application of various process discovery algorithms—**Alpha Miner**, **Inductive Miner**, and **Heuristic Miner**—different perspectives of the underlying process were visualized and analyzed. Performance comparison indicated that the **Inductive Miner** offered a balanced trade-off between model quality and discovery time.

Subsequently, the log was filtered and transformed into a format suitable for predictive modeling. A set of meaningful features was engineered to represent case-level characteristics, such as activity counts, case durations, and whether a payment occurred.

Using a **Random Forest Classifier**, the model achieved high predictive performance with perfect scores on the test set and a strong mean cross-validation score. This demonstrates the potential for **predictive process monitoring** to support real-time or early-intervention scenarios in traffic fine management.

The insights derived from this analysis can inform process optimization efforts, such as identifying bottlenecks or enhancing payment compliance. Furthermore, the framework developed here can serve as a foundation for extending the analysis with additional metrics, or different predictive targets.